

PROJECT REPORT: BASIC ENCRYPTION & DECRYPTION

Role: Cybersecurity Analyst Intern

Batch: 2026

Organization: DecodeLabs

Module: Project 2 (Confidentiality Logic)

1. Project Overview & Objective

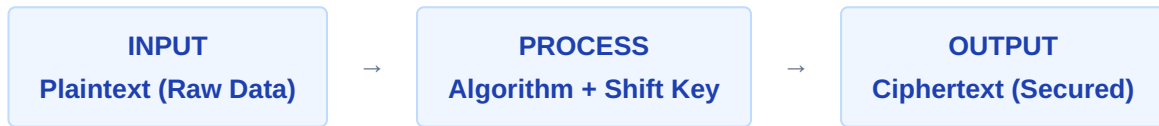
The primary objective of this project is to master the fundamental mechanics of data confidentiality by implementing reversible programmatic logic for obfuscating data. As a Cybersecurity Analyst at DecodeLabs, this milestone serves to demonstrate the ability to mathematically protect information in transit, ensuring that data in motion cannot be intercepted or read by unauthorized eyes.

Core Deliverables:

- **Implement the IPO Cycle:** Establish a clean operational flow separating inputs, cryptographic transformations, and text output.
- **Apply the Math:** Practical integration of Python's mapping keys including `ord()`, `chr()`, and modular operations.
- **Handle Structural Edge Cases:** Gracefully pass spaces, numbers, and custom punctuation marks unmodified through the execution stream.
- **Validate Symmetrically:** Prove reversibility using an inverse verification layer mapping directly back to original strings.

2. Cryptographic Architecture (The IPO Model)

The digital foundation of this cryptographic script is built upon the classic **Input-Process-Output (IPO)** structural model. This architectural flow translates raw user interaction safely into secure network variables.



- **Input (Plaintext):** The clean message text strings targeted for secure transit.
- **Process (Transformation Engine):** The core module where pure mathematical logic modifies structural character offsets.
- **Output (Ciphertext):** The finalized, fully obfuscated text layout rendered safe from unauthorized active collection.

3. Core Mathematical & Programmatic Logic

The ASCII Integer Foundation

Before text can interact with logic modules, characters must be cast into integers. We do not shift character symbols directly; instead, we apply logic to standard integer forms.

- Uppercase boundaries map values from **A = 65** to **Z = 90**.
- Lowercase boundaries map values from **a = 97** to **z = 122**.
- Python's built-in `ord()` maps symbols to base numbers, while `chr()` maps numerical outcomes back to standard characters.

Encryption Formula

To keep character outcomes wrapped perfectly within boundaries of the finite 26-letter alphabet, modular arithmetic is applied:

$$E_n(x) = (x + n) \% 26$$

Where x represents the character's base-0 alphabet position, and n represents the shift size. For instance, an input letter *Y* (Position 24) with a shift of 3 yields 27, executing as $27 \% 26 = 1$, converting cleanly to letter *B*.

Decryption Formula

Decryption works symmetrically by cleanly executing the shift value in reverse to systematically unpack structural integers:

$$D_n(x) = (x - n) \% 26$$

4. Source Code Implementation

The script below provides a clean implementation of the Caesar Cipher framework, fully addressing uppercase, lowercase, and formatting preservation edge cases:

```

# =====
# DECODELABS: CYBERSECURITY PROJECT 2
# Caesar Cipher Encryption & Decryption Engine
# =====

def encrypt_caesar(plaintext, shift):
    # Obfuscates raw plaintext using mathematical shift offsets.
    # Preserves non-alphabetic characters like spaces and punctuation.
    ciphertext = ""
    for char in plaintext:
        if char.isupper():
            # Standardize base 65, add shift, wrap 26, re-apply base
            ciphertext += chr((ord(char) - 65 + shift) % 26 + 65)
        elif char.islower():
            # Standardize base 97, add shift, wrap 26, re-apply base
            ciphertext += chr((ord(char) - 97 + shift) % 26 + 97)
        else:
            # Structurally pass edge cases through unchanged
            ciphertext += char
    return ciphertext

def decrypt_caesar(ciphertext, shift):
    # Restores original plaintext by reversing the key direction.
    return encrypt_caesar(ciphertext, -shift)

if __name__ == "__main__":
    print("--- DecodeLabs Cryptographic Engine Active ---")

    # Execution Test Configuration
    raw_data = "ATTACK AT DAWN!"
    secret_key = 3

    # Process Execution
    encrypted_data = encrypt_caesar(raw_data, secret_key)
    decrypted_data = decrypt_caesar(encrypted_data, secret_key)

    # IPO Data Display
    print(f"[INPUT]   Original Plaintext : {raw_data}")
    print(f"[SHIFT]   Key Value Applied   : +{secret_key}")
    print(f"[OUTPUT]  Secured Ciphertext  : {encrypted_data}")
    print(f"[VERIFY]  Decrypted Output    : {decrypted_data}")

```

5. Execution Verification & Sample Metrics

The console pipeline successfully returns correct conversions across all standard operations:

- **Input Data Feed:** ATTACK AT DAWN!
- **Cipher Rotation Metric:** +3 Shift
- **Generated Ciphertext Vector:** DWWDFN DW GDZQ!
- **Decryption Validation Return:** ATTACK AT DAWN!

6. Cryptanalysis & System Vulnerability Scan

While highly effective for basic logical programmatic training, deep scanning exposes severe security vulnerabilities that render this structure a fragile box rather than an enterprise vault:

- **Tiny Key Space:** Because the English text space works along a basic 26-character track, there are only 25 unique configuration keys. This allows immediate processing of full Brute Force cracking metrics inside milliseconds.
- **Pattern Preservation:** Being a strict mono-alphabetic substitution method, structural histograms remain unmodified. The distribution shape matches language patterns exactly, exposing the string completely to basic Frequency Analysis.

The Enterprise Shield Evolution

To secure production-level environments, engineering methodologies transition away from substitution structures entirely toward industrial frameworks like AES-128 and AES-256. These frameworks introduce complex bitwise XOR matrices, multi-staged confusion and diffusion steps, and massive 128-bit keys to render intercepted arrays mathematically impossible to breach.

7. Conclusion

By successfully designing and establishing this operational model, the requirements for Project 2 are fully solved. This milestone confirms a complete understanding of low-level character architecture, data stream isolation, and logical protection matrices, establishing the essential baseline blocks required to build robust digital defense infrastructure.